

Spotter Backend — Loom Demo Script

Fuel Route Optimizer | 5:00 Total

SECTION 1 — 0:00–0:30 | Introduction

WHAT TO SHOW ON SCREEN

Open VS Code with the project open. Show the file tree and open README.md.

WHAT TO SAY

“Hi everyone, my name is Aman, and this is my submission for the Spotter Backend Engineer assessment — the Fuel Route Optimizer.

The goal of this project is to build a robust Django REST Framework backend that accepts any start and finish location in the United States, computes the optimal driving route, and selects the most cost-effective fuel stops from the provided OPIS dataset.

The vehicle operates under realistic constraints: a maximum range of 500 miles on a full tank, fuel efficiency of 10 miles per gallon, and the assumption that the truck begins its journey with a full 50-gallon tank.

I've also built a clean, modular architecture that makes only a single call to the external routing engine and handles everything else locally with sub-second response times.”

SECTION 2 — 0:30–1:30 | Codebase & Architecture Overview

WHAT TO SHOW ON SCREEN

Keep VS Code open and briefly click through:

1. fuel_optimizer/models.py
2. fuel_optimizer/services/routing.py
3. fuel_optimizer/services/route_matching.py
4. fuel_optimizer/services/fuel_optimizer.py
5. fuel_optimizer/views.py

WHAT TO SAY

models.py: “Our FuelStation model stores truck stop metadata, retail price as a Decimal to prevent floating-point precision issues, and latitude/longitude with compound database indexes on state, city, price, and coordinates.”

routing.py: “All external routing logic is cleanly isolated behind a RoutingService. It calls OSRM driving directions and returns GeoJSON line geometry, converting distance and duration into miles and minutes. Crucially, as requested by the assessment, the API makes exactly ONE external routing call per request.”

route_matching.py: “We filter the 8,000+ stations down to a 30-mile route corridor using spatial grid bucketing and Haversine projection, running in under 50 milliseconds.”

fuel_optimizer.py: “We implement the classic greedy gas-station algorithm.”

views.py: “The view functions purely as an orchestrator, keeping all business logic strictly within service classes.”

SECTION 3 — 1:30–3:30 | Live Postman API Demonstration

WHAT TO SHOW ON SCREEN

Switch to Postman. Show the collection “Spotter Fuel Optimizer API” and run the four requests below one by one.

REQUEST A — New York to Chicago (Trip > 500 Miles)

Click “1. New York to Chicago” → Send.

“Now let’s switch to Postman to demonstrate the API.

Our first request is from New York to Chicago. When I click Send, notice how fast the response returns.

The route distance is 790.5 miles with full GeoJSON geometry. Because the truck leaves New York with a 500-mile full tank, it only needs two strategic fuel stops: Sheetz in Youngstown, Ohio at \$3.06 per gallon, and S&G; in Toledo at \$3.01. The response gives us exact sequence numbers, gallons purchased, stop fuel costs, and a fuel summary totaling \$87.68.”

REQUEST B — Los Angeles to Las Vegas (Trip ≤ 500 Miles)

Click “2. Los Angeles to Las Vegas” → Send.

“Next, let’s test a route under 500 miles: Los Angeles to Las Vegas.

The route is 270.38 miles. Notice that fuel_stops returns an empty array, and the fuel cost is zero dollars. This honors the requirement that the truck departs with a full tank of fuel, so no mid-route refueling is needed.”

REQUEST C — New York to Los Angeles (Cross-Country Multi-Stop)

Click “3. New York to Los Angeles” → Send.

“Now let’s test a true long-distance cross-country trip: New York to Los Angeles.

This is a 2,794-mile route spanning the entire continent. The optimizer calculates 12 optimal fuel stops. Most importantly: if we inspect the distance between each consecutive stop, every single driving leg is strictly under 500 miles, completely satisfying the vehicle range constraint.”

REQUEST D — Error Handling (Location Outside USA)

Click “5. Error - Location Outside USA” → Send.

“We also have robust validation. If a user supplies a location outside the United States, like ‘London, UK’, the API cleanly rejects it with HTTP 400 and a structured NON_USA_LOCATION error code without leaking any internal stack traces.”

SECTION 4 — 3:30–4:30 | Optimization Algorithm Explanation

WHAT TO SHOW ON SCREEN

Switch back to VS Code. Open fuel_optimizer/services/fuel_optimizer.py and scroll to the _run_greedy_optimizer method (around line 125).

WHAT TO SAY

“Let’s take a minute to explain the optimization algorithm in fuel_optimizer.py.

The core challenge is not simply finding the cheapest gas station in the country, but finding a feasible sequence of affordable stops where no leg exceeds 500 miles.

Here is how our greedy optimizer solves this:

1. We start at the origin with a full tank of 50 gallons.
2. From the current stop, we look ahead at all reachable stations within our 500-mile vehicle range.
3. If there is a cheaper station ahead within reach, we purchase only enough fuel to reach that cheaper station. This prevents buying expensive gas when cheaper gas is available just down the road.
4. If the current station is cheaper than all reachable stations ahead, it's the local minimum, so we fill the tank completely to 50 gallons and then proceed to the lowest-cost reachable station.
5. Finally, if there is ever a gap along the route greater than 500 miles with no stations, the system throws a `NoFeasibleFuelPlanError` returning HTTP 422."

SECTION 5 — 4:30–5:00 | Automated Tests, Deployment & Conclusion

WHAT TO SHOW ON SCREEN

Open the integrated terminal in VS Code. Run: `.\env\Scripts\pytest`. Let the terminal show the 34 passed in ~0.7 seconds output. Optionally show Render dashboard or the live URL.

WHAT TO SAY

"To ensure reliability, I've written a comprehensive automated test suite with pytest.

All 34 unit and integration tests pass in under one second. They test math formulas, boundary conditions, edge cases, CSV idempotency, and API contracts with fully mocked external services.

I have also committed all setup scripts, environment templates, and the Postman collection to the repository. The application is completely containerized with Docker and ready for 1-click deployment on Render.

Thank you for reviewing my assessment, and I look forward to discussing the implementation further!"